
THE AUSTRALIAN NATIONAL UNIVERSITY

Mid-Semester Examination – August 30th, 2022

COMP3320/6464/HONS HIGH PERFORMANCE SCIENTIFIC COMPUTATION

- Study period: 15 minutes
- Time allowed: 1 hour and 45 minutes (including submission time to Wattle)
- Permitted materials: open-book exam
- Exam questions total 100 marks
- All students answer all questions
- Clarity and conciseness in answers will be highly valued
- Marks may be lost for supplying irrelevant information

Topic 1: Floating point arithmetic [25 total marks]

- (a) Your machine uses a binary floating-point representation ($\beta = 2$) with precision $p = 4$, and whose smallest and largest exponents are $L = -3$ and $U = 4$, respectively.

Answer the following questions and justify your answer:

- (i) Define what normalization of a floating point is and why it is beneficial. [3 marks]
- (ii) What are the three smallest (non-zero) normalized positive numbers? [3 marks]
- (iii) What are the underflow and overflow limits? [2 marks]
- (iv) What are the subnormal numbers that can be represented by your machine? [2 marks]

- (b) The following code illustrates the effects of both approximation and rounding errors:

```
#include <stdio.h>
#include <math.h>
int main(void){
    float h,x;
    int i;
    x=12.2458;
    for (i=0, h=0.1; i<10; i++, h*=0.1)
        printf("%16.14f %8.1e %16.14f \n",
            1.0-x/sqrt(1+x*x),
            h,
            ((x+h-sqrt(1.0+(x+h)*(x+h))) - (x-sqrt(1.0+x*x)))/h);
    return 0;
}
```

It computes the derivative of $f(x) = x - \sqrt{1+x^2}$ both analytically and using the following finite difference method:

$$\frac{d(x-\sqrt{1+x^2})}{dx} = 1 - \frac{x}{\sqrt{1+x^2}} \approx \frac{(x+h)-\sqrt{1+(x+h)^2} - (x-\sqrt{1+x^2})}{h}$$

where h is the step size used in the finite difference method. The code uses $x=12.2458$ and various step sizes, yielding the following output:

```
0.00331767105871  1.0e-01  0.00329172679585
0.00331767105871  1.0e-02  0.00336073556360
0.00331767105871  1.0e-03  0.00381305491526
0.00331767105871  1.0e-04  0.00783510048370
0.00331767105871  1.0e-05  0.02224315692554
0.00331767105871  1.0e-06  0.33272002711545
0.00331767105871  1.0e-07  0.00000000000000
0.00331767105871  1.0e-08  0.00000000000000
0.00331767105871  1.0e-09  0.00000000000000
0.00331767105871  1.0e-10  0.00000000000000
```

- (i) Explain how this program illustrates both rounding error and approximation error. [5 marks]
- (ii) For any given value of h and without changing the value of h , state how you would modify the code to: (A) Decrease the effect of rounding errors; (B) Decrease the effect of approximation errors.

[5 marks]

- (iii) Write the expression for the relative error of the numerical derivative and discuss a way to evaluate both the rounding error and approximation errors for $h = 10^{-6}$

[5 marks]

Topic 2: CPU architecture [25 total marks]

(a) Name three different architectural enhancements that have been applied to latency-oriented (serial) CPUs to improve performance. Briefly describe how they work. **[6 marks]**

(b) What is cache associativity and how can it affect code performance? **[5 marks]**

(c) You are running on a machine with:

- Load/store latency of 2 cycles to L1 cache.
- Floating point add or multiply latency of 2 cycles.
- Superscalar architecture capable of one load, one store, one floating point addition, one floating point multiplication and 2 integer operations per clock cycle.

Consider the following loop in which `a` is a floating-point array.

```
for (i=0; i < 2*N; i++){  
    a[i+2] += a[i] * a[i+1];  
}
```

Assume that there is no pointer aliasing nor loop overhead (i.e., branching and increment of counter `i` are for free).

- (i) What is the minimum number of clock cycles that it will take to execute one loop iteration, when the compiler has not performed loop unrolling and pipelining? Give an instruction schedule, showing every clock cycle as a separate line, to explain your answer.

[5 marks]

- (ii) What is the speedup achieved when the loop is unrolled 2 times? Include the new code (in the format included above) and a new instruction schedule in your answer.

[5 marks]

- (iii) What is the speedup achieved with respect to the number of cycles required by (i) when load instructions and floating-point instructions are accelerated by a pipeline of depth 2? Justify your answer with an instruction schedule.

[5 marks]

Topic 3: Memory Architecture [25 marks]

a) Your machine has a CPU with a 4 KB direct-mapped L1 cache. The L1 cache line size (block size) is 16 bytes. You run the following routine

```
typedef float array_t[N][N];

void smoother(array_t a, array_t b){
    int i, j;
    for(j = 0; j < N; j += 2){
        for(i = 0; i < N; i += 2){
            b[i][j] = (a[i][j] + a[i+1][j]
                      + a[i][j+1] + a[i+1][j+1])*0.25;
        }
    }
}
```

where N is large and `sizeof(float)=4`. The L1 cache line size (block size) is 16 bytes.

- (i) What are the approximate read and write miss rates? Note: the read miss rate is defined as the ratio between the number of read misses and the sum of the number of read misses and read hits. **[6 marks]**
- (ii) Rewrite the routine to improve performance without changing code functionality and provide the read and write miss rates of your optimized code. Carefully justify your answer. **[6 marks]**

(b) Processors use caches to improve their performance.

- (i) Provide one example of C code that does not take advantage of temporal locality. Provide a detailed motivation for your choice. **[3 marks]**

(c) Shown below are there is an algorithm used to initialize the velocity field values of a fluid on a discrete Cartesian grid at given time t .

```
struct four_velocity{
    double vx;
    double vy;
    double vz;
    float t;
};

struct four_velocity vfield[N][N];

int i, j;
for(i = N-1; i >= 0; i--){
    for(j = N-1; j >= 0; j--){
        vfield[i][j].vx = 0.0;
        vfield[i][j].vy = 0.0;
        vfield[i][j].vz = 1.0;
        vfield[i][j].t = 0.0;
    }
}
```

where N is large, $\text{sizeof}(\text{double})=8$, $\text{sizeof}(\text{float})=4$, and due to padding inserted by the compiler **$\text{sizeof}(\text{four_velocity})=32$** .

- (i) Discuss the write hit rate of the nested loops in the code above when run on a machine with a 16KB two-way associative L1 cache, with a cache line size of 32 bytes.

[4 marks]

- (ii) How could you improve the cache performance the code? Provide a short code snippet and justify your answer.

[6 marks]

Topic 4: Performance Measurement and Optimization [25 marks]

- (a) In computing two measures of time are mainly used to analyse the temporal performance of code. Name the two measures of time and discuss advantages and disadvantages of their use? **[5 marks]**

- (b) Provide a C code or a pseudo-code snippet that allows you to measure the memory bandwidth and to estimate size of each cache level. Justify your algorithm. **[10 marks]**

- (c) You are given the following C code.

```
double fplus(double a, double b){
    return a+b/a;
}
double fminus(double a, double b){
    return a-b/a;
}
void mycode(double *a, double *b, double *c){
    int i, j;
    int count;

    count=0;
    for (i=0; i< 2*N; i++){
        for (j=0; j < i*(i+1)/2; j++){
            if(i < N){
                c[j] = fplus(a[i], b[j][i]);
            } else{
                c[j] = fminus(a[i], b[j][i]);
            }
            count++;
        }
    }
    printf("end loop nest: %d \n", count);
}
```

- (i) Write general expressions for the scaling of the nested loops as a function of N . **[2 marks]**
- (ii) If $N = 7$, what is the value of `count` that is printed out? **[2 marks]**

(iii) Rewrite the code to improve performance without affecting correctness and discuss your optimizations. **[6 marks]**